

Softwares de Sistemas

Java Virtual Machine

SOFTWARE BÁSICO, TURMA A
PROF. MARCELO LADEIRA CIG/UNB

Baseado no livro "The Java Virtual Machine Specification", Second Edition, Tim Lindholm & Frank Yellin. Parte 3 de 3

Shift (Deslocamento)

Desempilha um valor topo da pilha, gira-o e empilha o resultado

..., *value1*, *value2* $\Rightarrow$..., *result*

result é calculado como deslocar *value1* por *s* bits. *s* é o valor dos 5 bits de ordem mais baixa de *value2*. Para *long*, *s* é o valor dos 6 bits de ordem mais baixa.

ishl ou *lshl* /* int ou long para esquerda */

ishl = 120 (0x78) e *lshl* = 121 (0x79)

ishr ou *lshr* /* int ou long para direita */

ishr = 122 (0x7a) e *lshr* = 123 (0x7b)

iushr ou *lushr* /* int ou long sem sinal direita */

iushr = 124 (0x7c) e *lushr* = 125 (0x7d)

■ Insere zeros à esquerda

Instruções Lógicas Bit a Bit

Desempilha dois valores do topo da pilha, aplica o operador lógico bit a bit e empilha o resultado. Opera com `int` ou `long`
..., *value1*, *value2* $\Rightarrow$..., *result*

- Bitwise AND (`iand` e `land`)
iand = 126 (0x7e)
land = 127 (0x7f)
- Bitwise OR (`ior` e `lor`)
ior = 128 (0x80)
lor = 129 (0x81)
- Bitwise exclusive OR (`ixor` e `lxor`)
ixor = 130 (0x82)
lxor = 131 (0x83)

Incremento em Variável Local

Incrementa variável local (identificada por um índice) com uma constante (com sinal). Não afeta a pilha de operandos

iinc = 132 (0x84)

unsigned byte

/* index para o vetor variável local */

signed byte

/* constante a somar à variável */

- Comentários

- A variável no vetor de variáveis locais deve ser do tipo **int**
- Pode ser combinado com **wide** para criar um índice de 16 bits e constante de 16 bits com sinal

Comparação de float e double

Desempilha *value2* e *value1* do topo da pilha, compara-os e armazena o result do tipo int ..., *value1*, *value2* $\Rightarrow$..., *result*

1 se *value1* > *value2*, 0 se *value1* = *value2* ou -1 se *value1* < *value2*

value1 e *value2* são float

fcmpl = 149 (0x95)

e

fcmpg = 150 (0x96)

value1 e *value2* são doubles

dcmpl = 151 (0x97)

e

dcmpg = 152 (0x98)

Se ao menos um é NaN

- *dcmpg* e *fcmpg* empilham 1
- *dcmpl* e *fcmpl* empilham -1

Comparação de long

..., *value1*, *value2* $\Rightarrow$..., *result*

lcmp = 148 (0x94)

Desempilha *value2* e *value1* do topo da pilha, compara-os e armazena o

result **do tipo** int

1 se *value1* > *value2*

0 se *value1* = *value2*

-1 se *value1* < *value2*

value1 e *value2* são long

Comparação de int com zero

..., *value1* ⇒ ...

If<cond> *branchbyte1 branchbyte2*

ifeq = 153 (0x99)

ifne = 154 (0x9a)

iflt = 155 (0x9b)

ifge = 156 (0x9c)

ifgt = 157 (0x9d)

ifle = 158 (0x9e)

Desempilha *value1* (do tipo *int*) do topo da pilha, compara com zero e desvia se resultar true de acordo com a operação

- O offset deve estar contido no método que contém a instrução de desvio.
 - Se retornar false, retoma o processamento na próxima instrução

Comparação de dois int

..., *value1*, *value2* ⇒ ...

If_icmp<cond> *branchbyte1* *branchbyte2*

if_icmpeq = 159 (0x9f) *if_icmpne* = 160 (0xa0) *if_icmplt* = 161 (0xa1)

if_icmpge = 162 (0xa2) *if_icmpgt* = 163 (0xa3) *if_icmple* = 164 (0xa4)

Desempilha os **inteiros value1** e **value2** do topo da pilha, compara-os e desvia se resultar **true** de acordo com a operação

- O offset deve estar contido no método que contém a instrução de desvio.
 - Se retornar false, retoma o processamento na próxima instrução

Comparação de dois objetos

..., *value1*, *value2* ⇒ ...

If_acmp<cond> *branchbyte1* *branchbyte2*

if_acmpeq = 165 (0xa5)

if_acmpne = 166 (0xa6)

Desempilha *value1* e *value2* do topo da pilha compara-os e desvia se resultar *true* de acordo com a operação. *value1* e *value2* são *referências*.

- O offset deve estar contido no método que contém a instrução de desvio.
 - Se retornar *false*, retoma o processamento na próxima instrução

Instruções de Conversão de Tipo: Promoção

..., *value* $\Rightarrow$..., *result*

int para long, float ou double

i2l = 133 (0x85)

i2f = 134 (0x86)

i2d = 135 (0x87)

long para float ou double

l2f = 137 (0x89)

l2d = 138 (0x8a)

float para double

f2d = 141 (0x8d)

Instruções de Conversão de Tipo: Estreitamento

..., *value* $\Rightarrow$..., *result*

- Despreza os bits superiores e **pode mudar o sinal**

int para byte, char ou short

i2b = 145 (0x91)

/* e estende para inteiro com sinal */

i2c = 146 (0x92)

/* e estende para inteiro sem sinal */

i2s = 147 (0x93)

/* e estende para inteiro com sinal */

long para int

l2i = 136 (0x88)

float para int ou long

f2i = 139 (0x8b) *f2l* = 140 (0x8c)

double para int, long or float

d2i = 142 (0x8e)

d2l = 143 (0x8f)

d2f = 144 (0x90)

Instruções de Criação e Manipulação de Objetos

Criar uma nova instância de classe

new

Criar um novo array

newarray, anewarray, multianewarray

Acessar fields de classes ou de instâncias de classes

getstatic, putstatic, getfield, putfield

Carregar um componente de um array na pilha de operandos

baload, caload, saload, iaload, laload, faload, daload, aaload.

Armazenar um valor da pilha de operandos como um componente de um array

bastore, castore, sastore, iastore, lastore, fastore, dastore, aastore.

Obter o comprimento de um array

arraylength.

Verificar propriedades de instâncias de classes ou arrays

instanceof, checkcast.

Instância de uma Classe

new indexbyte1 indexbyte2

187 (0xbb) /* opcode */

16 bits unsigned /* **index** – índice para o pool de constante */

■ Comentários

- use $(\text{indexbyte1} \ll 8) \mid \text{byte2}$ para construir um valor de 16 bits. Os bytes são considerados sem sinais.
- **index** aponta para um nome (referência simbólica).
- o tipo desse nome deve ser **classe**.
- memória para a nova instância é alocada a partir do coletor de lixo.
- as **variáveis de instância** são iniciadas para o valor default.
- a referência *objectref* para a instância é armazenada na pilha.
- operação na pilha de operandos

... $\Rightarrow$..., *objectref*

Criação e Manipulação de Objetos: Vetor

newarray atype

188 (0xbc) /* opcode */
byte /* atype – tipo do array a ser criado */

- Comentários

- Codificação do tipo do array

T_BOOLEAN	4	T_CHAR	5
T_FLOAT	6	T_DOUBLE	7
T_BYTE	8	T_SHORT	9
T_INT	10	T_LONG	11

- memória para um array do tipo atype com count elementos é alocada
 - count deve ser um int positivo
 - os elementos são iniciados com seus valores padrão
 - a referência arrayref para o array é armazenada na pilha
 - operação na pilha de operandos
... count ⇒ ..., arrayref

Criação e Manipulação de Objetos: Array

multianewarray indexbyte1 indexbyte2 dimensions

197 (0xc5) /* opcode */

16 bits unsigned /* index – índice para o pool de constante */

unsigned byte /* dimensions – número de dimensões */

■ Comentários

- use (indexbyte1 << 8) | byte2 para construir um valor de 16 bits. Os bytes são sem sinais.
- index aponta para um nome (referência simbólica).
- o tipo desse nome deve ser classe, array ou interface
- memória para um array desse tipo com count_i elementos na i-dimensão é alocada a
 - todos os count deve ser int positivo maiores ou iguais a um
- os elementos são iniciados com null
- a referência arrayref para o array é armazenada na pilha
- operação na pilha de operandos

... count1, [count2, ...] ⇒ ..., arrayref

Acessar Field de Classe

getstatic indexbyte1 indexbyte2

178 (0xb2) /* opcode */

16 bits unsigned /* **index** – índice para pool de constante */

■ Comentários

- use $(\text{indexbyte1} \ll 8) \mid \text{byte2}$ para construir um valor de 16 bits. Os bytes são considerados sem sinais.
- **index** aponta para um nome (referência simbólica) para um **field** que dá o nome e descritor do **field** assim como a referência simbólica para a classe ou interface que o contém.
- a classe ou interface é iniciada (se não já o foi) e o valor do **field** é obtido e inserido na pilha de operandos
- operação na pilha de operandos

... $\Rightarrow$..., *value*

Acessar Field de Classe

putstatic indexbyte1 indexbyte2

179 (0xb3) /* opcode */

16 bits unsigned /* **index – índice para pool de constante** */

■ Comentários

- use (indexbyte1 << 8) | byte2 para construir um valor de 16 bits. Os bytes são considerados sem sinais.
- **index** aponta para um nome (referência simbólica) para um **field** que dá o **nome e descritor** do field assim como a referência simbólica para a classe ou interface que o contém.
- **a classe ou interface é iniciada (se não já o foi)** e o valor do **field** é iniciado com **value** retirado da pilha de operandos
 - se o tipo do descritor do field for **boolean, byte, char, short ou int** então *value* deve ser um int.
 - se o tipo do descritor for **float, long ou double** então *value* deve ser um float, long ou double, respectivamente.
- **operação na pilha de operandos**
..., *value* ⇒ ...

Acessar Field de Instância

getfield indexbyte1 indexbyte2

180 (0xb4) /* opcode */

16 bits unsigned /* **index** – índice para pool de constante */

■ Comentários

- **Objectref**, referência para o objeto que contém o field, retirada da pilha de operandos.
- use (indexbyte1 << 8) | byte2 para construir um valor de 16 bits. Os bytes são considerados sem sinais.
- **index** aponta para um nome (referência simbólica) para um **field** que dá o **nome e descritor** do field assim como a referência simbólica para a classe que o contém.
- o valor do field é **obtido e inserido na pilha de operandos**.
- se **protected**, objectref deve ser para a classe atual ou uma de suas subclasses.
- operação na pilha de operandos

..., objectref $\Rightarrow$..., value

Acessar Field de Instância

putfield indexbyte1 indexbyte2

181 (0xb5) /* opcode */

16 bits unsigned /* **index** – índice para pool de constante */

■ Comentários

- **value** e **objectref** são retirados da pilha de operandos.
- $\text{use}(\text{indexbyte1} \ll 8) \mid \text{byte2}$ para construir um valor de 16 bits. Os bytes são considerados sem sinais.
- **index** aponta para um nome (referência simbólica) para um **field** que dá o **nome e descritor** do field assim como a referência simbólica para a classe que o contém.
- o valor do field é **iniciado com value retirado da pilha de operandos**
 - se o tipo do descritor do field for **boolean, byte, char, short ou int** então *value* deve ser um int.
 - se o tipo do descritor for **float, long ou double** então *value* deve ser um float, long ou double.
- se **protected**, *objectref* deve ser para a classe atual ou uma de suas subclasses.
- operação na pilha de operandos
..., objectref, value $\Rightarrow$...

Criação e Manipulação de Objetos

Carregar um componente de um array na pilha de operandos

baload, caload, saload, iaload, laload, faload, daload, aaload.

Armazenar um valor da pilha de operandos como um componente de um array

bastore, castore, sastore, iastore, lastore, fastore, dastore, aastore.

Obter o comprimento de um array

arraylength.

Verificar propriedades de instâncias de classes ou arrays

instanceof, checkcast.

Carregar um Elemento de um Array na Pilha

Acessa **um elemento de um array** no heap e o **copia para a pilha de operandos**

Essas instruções não possuem operandos

Operação da pilha

..., **arrayref**, **index** $\Rightarrow$..., **value**

Ação

value = arrayref [index]

Instrução	opcode	Tipo
aaload	0x32	reference
baload	0x33	byte
caload	0x34	char
saload	0x35	short
iaload	0x36	int
laload	0x37	long
faload	0x38	float
daload	0x39	double

Outras Instruções

Gerenciamento da pilha de operandos

- *pop, pop2, dup, dup2, dup_x1, dup2_x1*
 - Desempilha topo (cat 1): ..., *value* $\Rightarrow$...,
 - Desempilha topo (cat 2): ..., *value* $\Rightarrow$..., Se topos forem cat 1 ...*value1, value2* $\Rightarrow$...,
 - Duplica o topo (cat 1): ..., *value* $\Rightarrow$..., *value, value*
 - Duplica o topo (cat 2): ..., *value* $\Rightarrow$..., *value, value* .
 - Se topos forem cat 1 ..., *value2, value1* $\Rightarrow$..., *value2, value1, value2, value1*
 - Duplica o topo e insere dois slots abaixo (topos são cat 1)
..., *value2, value1* $\Rightarrow$..., *value1, value2, value1*
 - Duplica o(s) topo(s) e insere dois ou três slots abaixo
..., *cat1, cat2* $\Rightarrow$..., *cat2, cat1, cat2*
..., *value3, value2, value1* $\Rightarrow$..., *value2, value1, value3, value2, value1*
onde *value3, value2, value1* são cat 1

Outras Instruções

Gerenciamento da pilha de operandos

- *dup_x2, dup2_x2, swap*
 - **dup_x2** – Duplica o topo e insere dois ou três slots abaixo
 $\dots, \text{value3}, \text{value2}, \text{value1} \Rightarrow \dots, \text{value1}, \text{value3}, \text{value2}, \text{value1}$
onde *value3, value2, value1* são cat 1
 $\dots, \text{cat2}, \text{cat1} \Rightarrow \dots, \text{cat1}, \text{cat2}, \text{cat1}$
 - **dup2_x2** – Duplica o(s) topo(s) e insere dois, três ou quatro slots abaixo
 $\dots, \text{value4}, \text{value3}, \text{value2}, \text{value1} \Rightarrow \dots, \text{value2}, \text{value1}, \text{value4}, \text{value3}, \text{value2}, \text{value1}$
onde *value3, value2, value1* são cat 1
 $\dots, \text{value3}, \text{value2}, \text{cat2} \Rightarrow \dots, \text{cat2}, \text{value3}, \text{value2}, \text{cat2}$ onde *value3* e *value2* são cat 1
 $\dots, \text{cat2}, \text{value2}, \text{value1} \Rightarrow \dots, \text{value2}, \text{value1}, \text{cat2}, \text{value2}, \text{value1}$
onde *value2, value1* são cat 1
 $\dots, \text{value2}, \text{value1} \Rightarrow \dots, \text{value1}, \text{value2}, \text{value1}$ onde *value2* e *value1* são cat 2
 - **swap** – Troca os elementos do topo da pilha (cat 1 apenas)

Outras Instruções

Transferência de controle

- Desvio condicional
 - *ifeq, iflt, ifle, ifne, ifgt, ifge, ifnull, ifnonnull, if_icmpeq, if_icmpne, if_icmplt, if_icmpgt, if_icmple, if_icmpge, if_acmpeq, if_acmpne.*
- Desvio condicional composto
 - *tableswitch, lookupswitch.*
- Desvio incondicional
 - *goto, goto_w, jsr, jsr_w, ret.*

Outras Instruções

Chamada de métodos e instruções de retorno

- *invokevirtual*
 - Chama um método de um **objeto**. É a forma normal de chamar um método da **mesma classe do objeto**.
- *invokeinterface*
 - Chama um método que é **implementado por uma interface**.
- *invokespecial*
 - Chama um método de instância requerendo tratamento especial (**método de iniciação, método privado ou método de superclass**)
- *invokestatic*
 - Chama um **método de classe** (static).
- *Invokedynamic*
 - Chama um **método dinâmico** (requer um objeto)

Outras Instruções

Chamada de métodos e instruções de retorno

- *invokevirtual* = 182 (0xb6)

- **Formato**

invokevirtual *indexbyte1* *indexbyte2*

- **Operação**

..., *objectref*, [*arg1*, [*arg2* ...]] $\Rightarrow$...

- **Descrição**

- O **índice formado pelos dois bytes** apontam para uma entrada no Constant Pool que dá o **nome e o descritor do método sendo chamado**, o que **define o número de argumentos** que possui. O método não pode ser um método construtor. Esse método **deve ser da classe do objeto *objectref* ou de super classe**. Os argumentos são retirados da pilha e um frame é construído para executar esse método com os argumentos passados.

Sumário das Instruções da JVM

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	nop	aconst_null	iconst_m1	iconst_0	iconst_1	iconst_2	iconst_3	iconst_4	iconst_5	lconst_0	lconst_1	fconst_0	fconst_1	fconst_2	dconst_0	dconst_1
1	bipush	Sipush	ldc	ldc_w	ldc2_w	iload	lload	fload	dload	aload	iload_0	iload_1	iload_2	iload_3	lload_0	lload_1
2	lload_2	lload_3	fload_0	fload_1	fload_2	fload_3	dload_0	dload_1	dload_2	dload_3	aload_0	aload_1	aload_2	aload_3	iaload	laload
3	faload	Daload	aaload	baload	caload	saload	istore	lstore	fstore	dstore	astore	istore_0	istore_1	istore_2	istore_3	lstore_0
4	lstore_1	lstore_2	lstore_3	fstore_0	fstore_1	fstore_2	fstore_3	dstore_0	dstore_1	dstore_2	dstore_3	astore_0	astore_1	astore_2	astore_3	iastore
5	lastore	Fastore	dastore	aastore	bastore	castore	sastore	pop	pop2	dup	dup_x1	dup_x2	dup2	dup2_x1	dup2_x2	swap
6	iadd	ladd	fadd	dadd	isub	lsub	fsub	dsub	imul	lmul	fmul	dmul	idiv	ldiv	fdiv	ddiv
7	irem	lrem	frem	drem	ineg	lneg	fneg	dneg	ishl	lshl	ishr	lshr	iushr	lushr	iand	land

Sumário das Instruções da JVM

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
8	ior	lor	ixor	lxor	iinc	i2l	i2f	i2d	l2i	l2f	l2d	f2i	f2l	f2d	d2i	d2l
9	d2f	l2b	i2c	i2s	lcmp	fcmpl	fcmpg	dcmpl	dcmpg	ifeq	ifne	iflt	ifge	ifgt	ifle	if_icmpeq
a	if_icmpne	if_icmplt	if_icmpge	if_icmpgt	if_icmple	if_acmpeq	if_acmpne	goto	jsr	ret	table switch	lookup switch	ireturn	lreturn	freturn	dreturn
b	areturn	Return	get static	put static	getfield	putfield	invoke virtual	invoke special	invoke static	invoke interface		new	new array	anew array	array length	athrow
c	check cast	instance of	monitor enter	monitor exit	wide	multi anew array	ifnull	ifnonnull	goto_w	jsr_w	break point					
d																
e																
f															impdep 1	impdep 2

Os três opcode em destaque são reservados pela Sun.